OptiCrop: Smart Agricultural Production Optimization Engine

Project Description:

The project "Smart Agricultural Production Optimization Engine" (OptiCrop) aims to develop an advanced software system that utilizes data-driven insights to optimize agricultural production for different crops. By integrating key environmental factors such as Nitrogen (N), Phosphorous (P), Potassium (K) levels, soil temperature, humidity, pH, rainfall, and crop types, OptiCrop seeks to provide intelligent recommendations to farmers for maximizing yields and resource efficiency.

The knowledge and insights gained through the OptiCrop project can be utilized by agricultural researchers, policymakers, and stakeholders to advance the understanding of crop-environment interactions and inform the development of evidence-based agricultural strategies.

The OptiCrop project revolutionizes agricultural practices by providing farmers with a smart production optimization engine. By integrating key environmental factors and crop-specific recommendations, OptiCrop enables farmers to achieve optimal yields, improve resource efficiency, and contribute to sustainable agricultural practices.

Scenarios

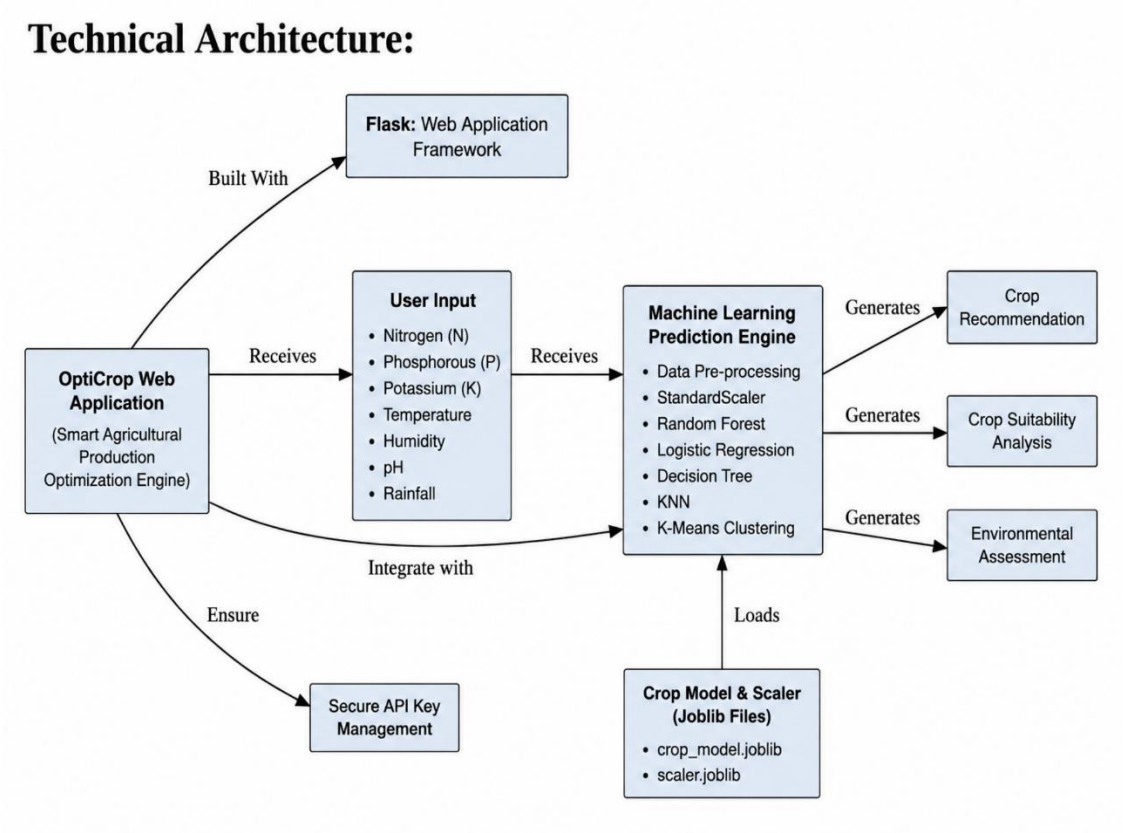
Scenario 1: Smart Crop Recommendation for Farmers A farmer enters soil and environmental details such as Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH level, and rainfall. The system analyzes the data and recommends the most suitable crop for maximum yield and better farming efficiency.

Scenario 2: Crop Suitability and Environmental Assessment A user wants to understand whether the current soil and climate conditions are suitable for a particular crop. The application evaluates the input parameters and provides insights about crop compatibility, environmental suitability, and productivity potential.

Scenario 3: Agricultural Research and Policy Planning An agricultural researcher or policymaker uses the system to analyze crop-environment relationships and identify patterns in agricultural production. The platform helps in making data-driven decisions for sustainable farming strategies and resource optimization.

Technical Architecture:

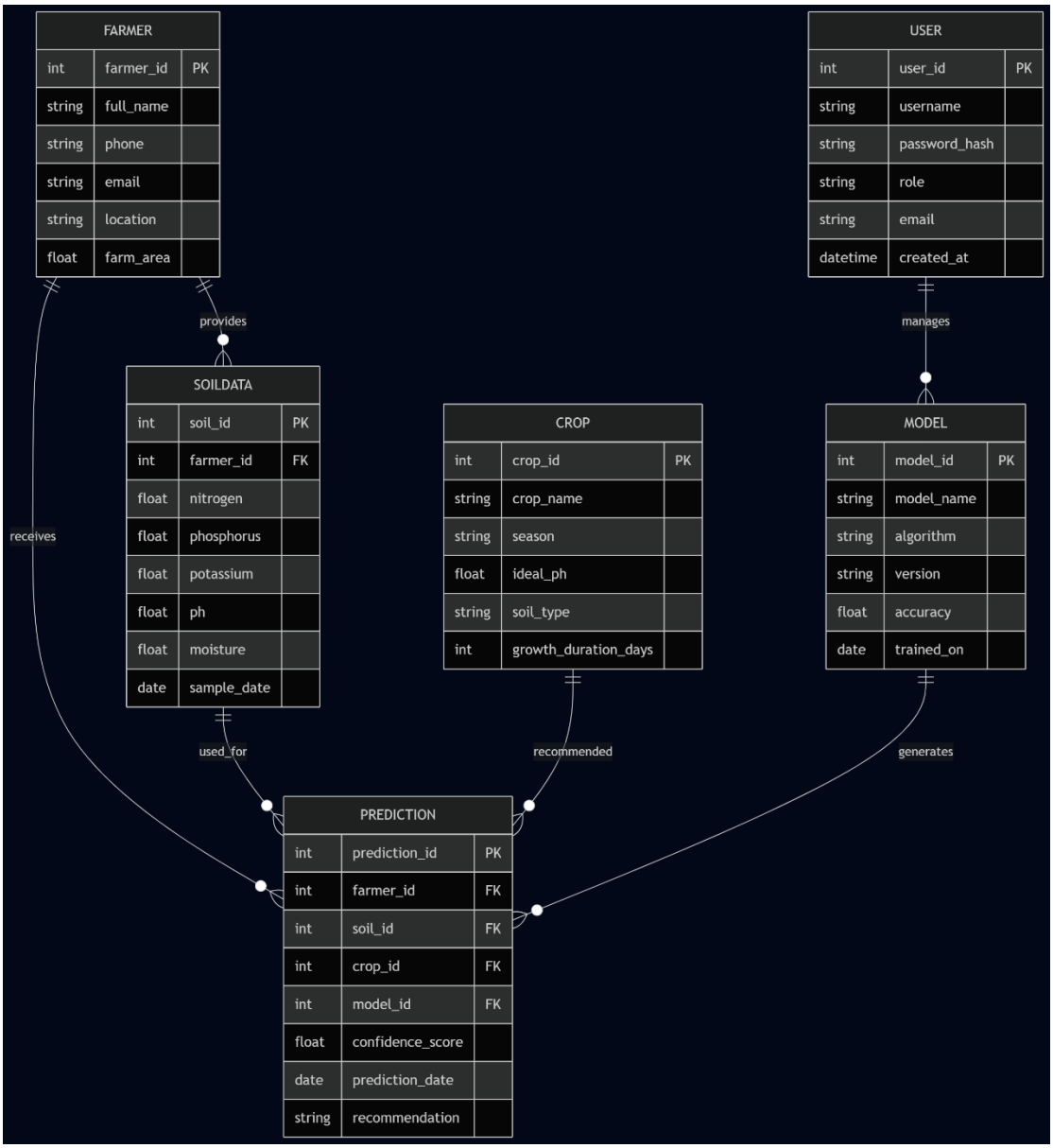
Technical Architecture:



Description: OptiCrop follows a modular, linear data pipeline architecture. Data flows from the raw dataset through cleaning and exploratory analysis, into a scaling and model-comparison stage, and finally into a Flask-based web application that serves predictions through an HTML/CSS/JavaScript frontend.

- Crop_recommendation.csv Dataset — the raw historical dataset of soil and climate records.
- Data Cleaning & Jupyter EDA — missing-value checks, outlier handling, and exploratory analysis.
- StandardScaler Scaling & train.py Comparison — feature scaling and multi-model comparison.
- Save Models & Scalers as Joblib files — serialization of the best-performing model.
- Flask Web Server (app.py) — backend that loads the model and serves predictions.
- HTML/CSS/JS Frontend Form UI — the user-facing interface that collects input and displays results.

Entity Relationship Diagram



The Entity Relationship Diagram illustrates the database structure by defining key entities and their relationships. It helps organize agricultural data efficiently and supports accurate prediction and optimization processes.

Pre-requisites:

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Storage: Minimum 10 GB free space
- Internet Connection for downloading datasets and packages

Software Requirements

- Operating System: Windows / Linux / macOS
- Python 3.x (3.10+ recommended)
- Anaconda Navigator
- Jupyter Notebook / Spyder
- Visual Studio Code or PyCharm
- Flask Framework

Skills Required

- NumPy
- Pandas
- Scikit-learn
- Matplotlib (Python Package)
- SciPy
- Seaborn
- Flask (Web Framework)
- Machine Learning

Mentor

No mentor assigned yet.

Team Members

Name	Role
Dasari Srividya	Team Lead
Devarakonda Revathi	Member
Idamakanti Anusha	Member
Chokkani Ekshith	Member
Shaik Khaadar vali	Member

Table 1: Project Team Members

Project Stats

- Total Epics: 9
- Total Tasks: 25
- Total Subtasks: 0

Project Workflow:

Milestone 1: Environment Setup and Core Software Installation

- Activity 1.1: Install Python (Version 3.10+) and configure the system PATH.
- Activity 1.2: Install Git for version control.
- Activity 1.3: Install Visual Studio Code and the Python/Jupyter extensions.
- Activity 1.4: Create the project workspace and install required Python libraries.

Milestone 2: Dataset Understanding and Data Pre-processing

- Activity 2.1: Understand the role and structure of the Crop Recommendation dataset.
- Activity 2.2: Perform Exploratory Data Analysis (EDA) using Pandas, Matplotlib, and Seaborn.
- Activity 2.3: Handle missing values and outliers, and extract seasonal crop groupings.
- Activity 2.4: Split and scale the data using train_test_split and StandardScaler.

Milestone 3: Model Building and Evaluation

- Activity 3.1: Apply K-Means Clustering and determine the optimal cluster count using the Elbow Method.
- Activity 3.2: Implement Logistic Regression for crop prediction.
- Activity 3.3: Build the train.py script to compare Logistic Regression, KNN, Decision Tree, and Random Forest models.
- Activity 3.4: Evaluate model performance and serialize the best model using joblib.
- Activity 3.5: Predict the best crop based on given input parameters.

Milestone 4: Application Building

- Activity 4.1: Build the Flask backend (app.py) to load the model and expose the /predict route.
- Activity 4.2: Build the HTML, CSS, and JavaScript frontend for user interaction.

Milestone 5: Deployment and Testing

- Activity 5.1: Run the training pipeline and verify serialized model output.
- Activity 5.2: Run the Flask development server and access the application locally.
- Activity 5.3: Perform manual verification using sample test scenarios.
- Activity 5.4: Diagnose and resolve common runtime issues.

Milestone 6: Conclusion

Summarizes the key work completed, highlights the main outcomes and findings, and reflects on the project's overall success and future scope.

Milestone 1: Environment Setup and Core Software Installation

In this milestone, we prepare the computer with the correct programming tools. Setting this up correctly is the most important step to prevent compatibility and configuration errors later.

Activity 1.1: Installing Python (Version 3.10+)

Python is the industry-standard language for machine learning.

- Download the installer from the official Python Downloads page.
- CRITICAL STEP: Run the installer and check the box that says “Add Python to PATH” before clicking Install. If you skip this, python commands will not work in your terminal.
- Verify the installation by opening your terminal and running the version check command.

```
python --version
```

Why we use Python: Python provides a mature ecosystem of libraries (like Scikit-Learn and Pandas) written in optimized C. This allows heavy numerical matrix operations to be performed easily without writing low-level code.

Activity 1.2: Installing Git

Git is a version control system used to track code history and back up the project.

- Download the installer from the official Git website.
- Run the installer and keep the default options selected.
- Verify by opening the terminal and checking the Git version.

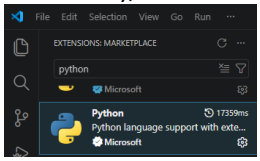
```
git --version
```

Why we use Git: Version control acts as a safety net. As you write and experiment with model code, it is easy to accidentally break dependencies. Git allows you to save snapshots (commits) and roll back your code to a working state in seconds.

Activity 1.3: Installing Visual Studio Code (VS Code)

VS Code is the primary editor used to write scripts and notebooks.

- Download and run the installer from the VS Code website.
- Open VS Code, click the Extensions icon on the left sidebar, search for Python and Jupyter (both by Microsoft), and install them.



Why we use VS Code: VS Code bridges the gap between interactive exploration (Jupyter Notebooks) and writing production-ready scripts (train.py), allowing the entire development lifecycle to be managed in a single editor.

Activity 1.4: Creating the Project Workspace and Installing Libraries

A structured folder layout ensures raw data remains isolated, serialized models are stored securely, and web backend assets are cleanly separated. Create a folder named opticrop and organize it as follows:

```
opticrop/
├── data/
│   └── Crop_recommendation.csv
├── models/
│   ├── crop_model.joblib
│   └── scaler.joblib
├── templates/
│   └── index.html
├── static/
│   ├── style.css
│   └── script.js
├── notebook.ipynb
└── app.py
```

Note: Whenever you write or paste code into any file in VS Code, remember to press Ctrl + S (Windows) or Cmd + S (macOS) to save your changes. If you do not save, the file remains empty and your code will not run.

Installing Required Libraries

Open a terminal inside the project folder and run the installation command:

```
pip install numpy pandas matplotlib seaborn scikit-learn joblib flask
```

- pandas: Serves as a "Virtual Excel". It loads, merges, and cleans tabular CSV data.
- numpy: Handles low-level mathematical operations on multi-dimensional matrix arrays.
- matplotlib & seaborn: Generate statistical charts and correlation heatmaps.
- scikit-learn: Houses the classifiers, data splitters, and validation metrics.
- joblib: Saves trained model weights into a portable binary file.
- flask: Runs the lightweight local backend application server.

Milestone 2: Dataset Understanding and Data Pre-processing

Activity 2.1: Understanding the Role of the Dataset

A machine learning model has no innate human intelligence — it relies entirely on historical evidence. The Crop Recommendation dataset contains details of 2,200 previous agricultural yields, each labeled with the crop that performed best under those specific conditions. The model analyzes these historical records, learns which traits correlate with crop types, and uses those patterns to recommend crops to farmers in real time.

Note: "Garbage In, Garbage Out" — the accuracy and fairness of the model depend entirely on the quality and size of the dataset.

Dataset Columns Explained

Column	Description
N (Nitrogen)	Nitrogen content ratio in the soil (mg/kg). Biological role: the "leaf builder," helping plants grow healthy green foliage.
P (Phosphorous)	Phosphorous content ratio in the soil (mg/kg). Biological role: the "root maker," stimulating early root development and flowering.
K (Potassium)	Potassium content ratio in the soil (mg/kg). Biological role: the "defense helper," regulating cell processes, disease resistance, and water intake.
temperature	Air temperature in Celsius (°C).
humidity	Relative air humidity in percentage (%). High humidity reduces plant evaporation.
ph	Soil pH level, ranging from 0.0 to 14.0, where 7.0 is neutral.
rainfall	Total regional rainfall volume in millimeters (mm).
label	The crop that succeeded in these conditions (e.g. rice, maize, mango, banana, coffee) — the target variable.

Table 2: Dataset Column Descriptions

Activity 2.2: Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is the first phase of any data science project. It allows the team to view basic statistical metrics, check distributions, look for missing fields, and visualize relationships between features using correlation matrices.

Ingesting the Soil Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
df = pd.read_csv('data/Crop_recommendation.csv')
print("Dataset Loaded Successfully!")
print(f"Dataset Shape: {df.shape} (Rows, Columns)")
print("\nFirst 5 Records:")
print(df.head())
```

- import pandas as pd: imports Pandas, Python's version of Microsoft Excel for tabular DataFrames.
- df = pd.read_csv(...): loads the raw spreadsheet into memory as a DataFrame named df.
- df.shape: returns (2200, 8) — 2,200 historical crop records and 8 columns.
- df.head(): displays the top 5 rows for visual inspection.

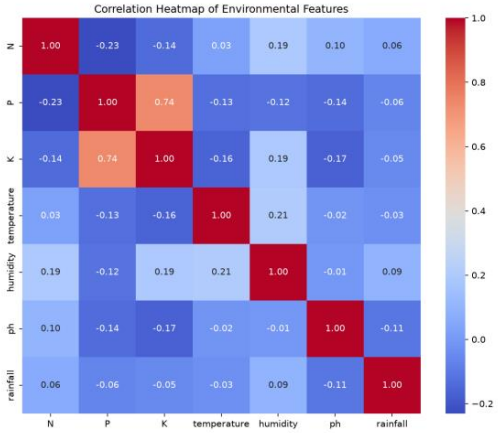
Statistical Summaries and Quality Audits

```
print("Data Summary Statistics:")
print(df.describe())
print("\nMissing Values per Feature:")
print(df.isnull().sum())
print("\nTarget Class Distribution (Unique Crops):")
print(df['label'].value_counts())
```

- df.describe(): generates mean, standard deviation, min, max, and quartiles for every numeric column. Average temperature is around 25.6°C, and average soil pH is 6.47 (slightly acidic, normal for agricultural land).
- df.isnull().sum(): checks for empty cells (NaN values); machine learning algorithms cannot process blank values.
- df['label'].value_counts(): confirms exactly 100 records for each of the 22 unique crops — a perfectly balanced dataset.

Correlation Heatmap of Soil and Weather Factors

```
plt.figure(figsize=(10, 8))
numeric_cols = df.select_dtypes(include=[np.number])
sns.heatmap(numeric_cols.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Heatmap of Environmental Features')
plt.show()
```



numeric_cols.corr() computes the Pearson correlation coefficient matrix — values near 1.0 indicate strong positive correlation, while 0 means no connection. In the soil data, Nitrogen, Phosphorus, and Potassium show relatively low correlations with each other, meaning they act as independent nutrients in the soil.

Activity 2.3: Handling Missing Values, Outliers, and Seasonal Extraction

The dataset is examined for missing and null values before preprocessing. Functions such as df.shape, df.info(), and df.isnull().sum() are used to analyze dataset structure, data types, and null records, ensuring clean and reliable data for accurate crop prediction.

Handling Outliers

Outliers are identified visually using boxplots. Upper and lower bounds are calculated mathematically using the Interquartile Range (IQR):

- Upper Bound = Q3 + 1.5 × IQR
- Lower Bound = Q1 − 1.5 × IQR

A boxplot reveals that the Potassium feature contains outliers. Log transformation is then applied to normalize its distribution.

Extracting Seasonal Crops

Crops are grouped and analyzed based on seasonal environmental conditions such as temperature, humidity, and rainfall. This process helps identify suitable crops for summer, winter, and rainy seasons to improve prediction accuracy and generate season-based agricultural recommendations.

Activity 2.4: Splitting and Scaling the Data

The dataset is divided into feature variables (X) and target variable (y) for machine learning model training. The train_test_split() function from Scikit-learn is used to split the dataset into training and testing sets with appropriate test size and random state values.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = df.drop(columns=['label'])
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

- train_test_split(..., test_size=0.2, stratify=y): the model trains on 80% of records (1,760 rows) and reserves 20% (440 rows) to test predictive accuracy on unseen soils.
- StandardScaler(): normalizes feature scales so that features with larger raw values (e.g. rainfall) are not treated as more important than features with smaller raw values (e.g. pH).

Note: The scaler is fit only on the training split (X_train) and then applied to both splits. Fitting on the test set would leak statistical information from the unseen test set into the training loop (data leakage).

Milestone 3: Model Building and Evaluation

This milestone trains and compares multiple classification models to find the one that balances accuracy with minority-class detection, then serializes the winning model for deployment.

Activity 3.1: K-Means Clustering

The K-Means clustering algorithm is applied to group similar agricultural patterns based on environmental features. The Elbow Method is used to determine the optimal number of clusters by analyzing WCSS (Within-Cluster Sum of Squares) values. The model is trained using the fit() and fit_predict() methods to identify meaningful crop groupings for intelligent agricultural analysis.

The Elbow Graph plots the Within-Cluster Sum of Squares (WCSS) against different cluster values and identifies the point where the rate of decrease sharply changes, known as the “elbow point.”

Activity 3.2: Logistic Regression

The Logistic Regression algorithm is implemented for crop prediction using agricultural environmental features. The model is trained using the fit() method and predictions are generated using the predict() function. Model performance is evaluated to analyze prediction accuracy and crop recommendation capability.

Activity 3.3: Model Comparison Script (train.py)

We transition from interactive notebook exploration to a production-grade Python script named train.py. While Jupyter is excellent for visual exploration (EDA), standard Python scripts are preferred in production to run training pipelines end-to-end and save the resulting model files.

- Logistic Regression: a linear model that estimates the probability of crop suitability scores. Fast and simple.
- K-Nearest Neighbors (KNN): a distance-based classifier that looks at the 5 most similar historical soil records to vote on the crop.
- Decision Tree Classifier: a tree model that splits parameters recursively using simple True/False check paths.
- Random Forest Classifier: an ensemble model that builds 100 decision trees and averages their outputs. Highly robust, handles non-linear relationships, and reduces variance.
- K-Means Clustering: an unsupervised algorithm that groups similar soil and weather profiles into 5 macro categories, helping researchers spot regional patterns without using labels.

```
import os
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score, classification_report
import joblib
```

```
def run_training_pipeline():
    print("Step 1: Loading agricultural dataset...")
    df = pd.read_csv('data/Crop_recommendation.csv')

    print("Step 2: Resolving missing values...")
    for col in df.columns[:-1]:
        if df[col].isnull().sum() > 0:
            df[col] = df[col].fillna(df[col].median())

    X = df.drop(columns=['label'])
    y = df['label']
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y)

    print("Step 3: Applying Standard Scaling to features...")
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    print("\nStep 4: Training and comparing classification models...")
    models = {
        "Logistic Regression": LogisticRegression(max_iter=1500, random_state=42),
        "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=5),
        "Decision Tree": DecisionTreeClassifier(random_state=42),
        "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42)
    }

    best_acc = 0.0
    best_model = None
    best_model_name = ""

    for name, clf in models.items():
        clf.fit(X_train_scaled, y_train)
        y_pred = clf.predict(X_test_scaled)
        acc = accuracy_score(y_test, y_pred)
        print(f"-> {name} Test Accuracy: {acc * 100:.2f}%")
        if acc > best_acc:
            best_acc = acc
            best_model = clf
            best_model_name = name

    kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
    kmeans.fit(X_train_scaled)
    print("\nUnsupervised K-Means clustering completed on scaled features.")
    print(f"\nWinner Selected: {best_model_name} with {best_acc * 100:.2f}% accuracy.")

    winner_preds = best_model.predict(X_test_scaled)
    print("\nWinner Performance Report:")
    print(classification_report(y_test, winner_preds))

    print("Step 5: Serializing and saving best model & scaler...")
    os.makedirs('models', exist_ok=True)
    joblib.dump(best_model, 'models/crop_model.joblib')
    joblib.dump(scaler, 'models/scaler.joblib')
    print("✓ Output model: models/crop_model.joblib")
    print("✓ Output scaler: models/scaler.joblib")
```

```
if __name__ == '__main__':
    run_training_pipeline()
```

Execute the training script from the project terminal:

```
python train.py
```

Activity 3.4: Evaluating Model Performance and Saving the Best Model

All models are evaluated and compared using classification metrics such as Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and ROC-AUC Score. After evaluation, the best-performing model is selected based on its overall classification results and generalization capability, then serialized using joblib.dump() and stored as crop_model.joblib for reuse in the Flask application.

Model	Test Accuracy
Logistic Regression	96.59%
K-Nearest Neighbors	97.95%
Decision Tree	98.64%
Random Forest (Winner)	99.32%

Table 3: Model Accuracy Comparison Results

Activity 3.5: Predicting the Best Crop Based on Given Parameters

The saved model is utilized to predict the most suitable crop based on the environmental and soil conditions provided by the user. Input features are collected from the user interface and passed to the trained machine learning model using model.predict(). The model processes these parameters and generates a crop recommendation best suited for the given conditions, helping farmers make data-driven decisions for improving crop productivity and efficient resource utilization.

Milestone 4: Application Building

This milestone focuses on developing the web application and integrating it with the trained machine learning model. The system accepts user inputs, generates crop predictions, and displays recommendations through an interactive interface.

Activity 4.1: Building the Flask Backend (app.py)

API Routing maps network URLs (endpoints) to specific Python functions using @app.route(). JavaScript on the frontend captures user entries and packages them as JSON; the backend extracts this JSON via request.get_json(), converts the values to floats, and packs them into a NumPy array matching the exact structure used during model training. The Flask server loads the serialized model weights with joblib.load() and, on each request, calls model.predict() to return the recommended crop label to the client.

Running the model on a dedicated backend server (rather than in browser JavaScript) keeps the client lightweight and secure, since machine learning models can weigh several hundred megabytes.

```
from flask import Flask, request, jsonify, render_template
import joblib
import numpy as np
import os

app = Flask(__name__)

MODEL_PATH = 'models/crop_model.joblib'
SCALER_PATH = 'models/scaler.joblib'

if not os.path.exists(MODEL_PATH) or not os.path.exists(SCALER_PATH):
    print("ERROR: Model or Scaler not found! Please run train.py first.")
    exit(1)

model = joblib.load(MODEL_PATH)
scaler = joblib.load(SCALER_PATH)

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    try:
        data = request.get_json()
        # Build features list in the exact order the model expects:
        # N, P, K, temperature, humidity, ph, rainfall
        features_raw = np.array([[
            float(data['N']), float(data['P']), float(data['K']),
            float(data['temperature']), float(data['humidity']),
            float(data['ph']), float(data['rainfall'])
        ]])

        # Apply boundary checks
        n, p, k, temp, hum, ph, rain = features_raw[0]
        if not (0 <= n <= 250):
            return jsonify({'error': 'Nitrogen must be between 0 and 250 mg/kg.'}), 400
        # ... additional boundary checks for P, K, temperature, humidity, pH, rainfall

        scaled = scaler.transform(features_raw)
        prediction = model.predict(scaled)
        return jsonify({'success': True, 'crop': prediction[0]})
    except Exception as e:
        return jsonify({'error': str(e)}), 400

if __name__ == '__main__':
    app.run(debug=True)
```

[TO BE FILLED: The complete boundary-validation logic for P, K, temperature, humidity, pH, and rainfall was not fully specified in the source material — only the Nitrogen check and the pH out-of-bounds test scenario were provided.]

Activity 4.2: Building the Frontend (HTML, CSS, JavaScript)

Web applications operate on a Client-Server Architecture. The client (index.html, style.css, script.js) collects information from the user and displays results. The server (app.py) listens for incoming HTTP requests, feeds features to the machine learning model, and returns the prediction. When the user submits the form, JavaScript packages the inputs into a JSON dictionary and sends it via an HTTP POST request to the /predict route. The page then updates dynamically based on the response, without a full reload.

Structure, style, and behavior are separated into HTML, CSS, and JavaScript respectively, following the Separation of Concerns principle, making it easy to redesign the UI or tweak frontend validations without editing backend code.

Frontend Template (templates/index.html)

The homepage contains a crop-prediction form collecting Nitrogen, Phosphorous, Potassium, temperature, humidity, pH, and rainfall values, along with a submit button and a result display area.

```
<form id="predictionForm">
  <input type="number" step="any" id="N" required placeholder="0 - 250">
  <input type="number" step="any" id="P" required placeholder="0 - 250">
  <input type="number" step="any" id="K" required placeholder="0 - 300">
  <input type="number" step="any" id="temperature" required placeholder="e.g. 24.5">
  <input type="number" step="any" id="humidity" required placeholder="e.g. 78">
  <input type="number" step="any" id="ph" required placeholder="0.0 - 14.0">
  <input type="number" step="any" id="rainfall" required placeholder="e.g. 180">
  <button type="submit">Identify Best Crop</button>
</form>
<div id="result"></div>
```

Application Logic Script (static/script.js)

On form submission, script.js prevents the default page reload, packages the seven input values into a JSON object, and sends them via fetch() to the /predict endpoint. The response is parsed and displayed in a green success banner (with the recommended crop name) or a red error banner (with the validation message).

```
document.getElementById('predictionForm').addEventListener('submit', async (e) => {
  e.preventDefault();
  const data = {
    N: parseFloat(document.getElementById('N').value),
    P: parseFloat(document.getElementById('P').value),
    K: parseFloat(document.getElementById('K').value),
    temperature: parseFloat(document.getElementById('temperature').value),
    humidity: parseFloat(document.getElementById('humidity').value),
    ph: parseFloat(document.getElementById('ph').value),
    rainfall: parseFloat(document.getElementById('rainfall').value)
  };
  const response = await fetch('/predict', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data)
  });
  const res = await response.json();
  // Update #result with success (green) or error (red) banner
});
```

The stylesheet (static/style.css) implements a dark card-based layout with a green accent color for success states and a red accent for error states.

Milestone 5: Deployment and Testing

Activity 5.1: Running the Model Training Script

Before the server can process predictions, the serialized model and scaler weights must be generated. Open a terminal in the root of the opticrop directory and execute the training pipeline:

```
python train.py

Step 1: Loading agricultural dataset...
Step 2: Resolving missing values...
Step 3: Applying Standard Scaling to features...
Step 4: Training and comparing classification models...
-> Logistic Regression Test Accuracy: 96.59%
-> K-Nearest Neighbors Test Accuracy: 97.95%
-> Decision Tree Test Accuracy: 98.64%
-> Random Forest Test Accuracy: 99.32%
Unsupervised K-Means clustering completed on scaled features.
Winner Selected: Random Forest with 99.32% accuracy.
Step 5: Serializing and saving best model & scaler...
✓ Output model: models/crop_model.joblib
✓ Output scaler: models/scaler.joblib
```

Verify that crop_model.joblib and scaler.joblib have been successfully created inside the models/ directory.

Activity 5.2: Running the Flask Web Server

With the model weights and all frontend/backend files created, start the local development web server:

```
python app.py

Crop model and scaler loaded successfully!
 * Serving Flask app 'app'
 * Debug mode: on
 * Running on http://127.0.0.1:5000
```

Open a web browser and navigate to http://localhost:5000 to access the Crop Recommendation form.

Home Page

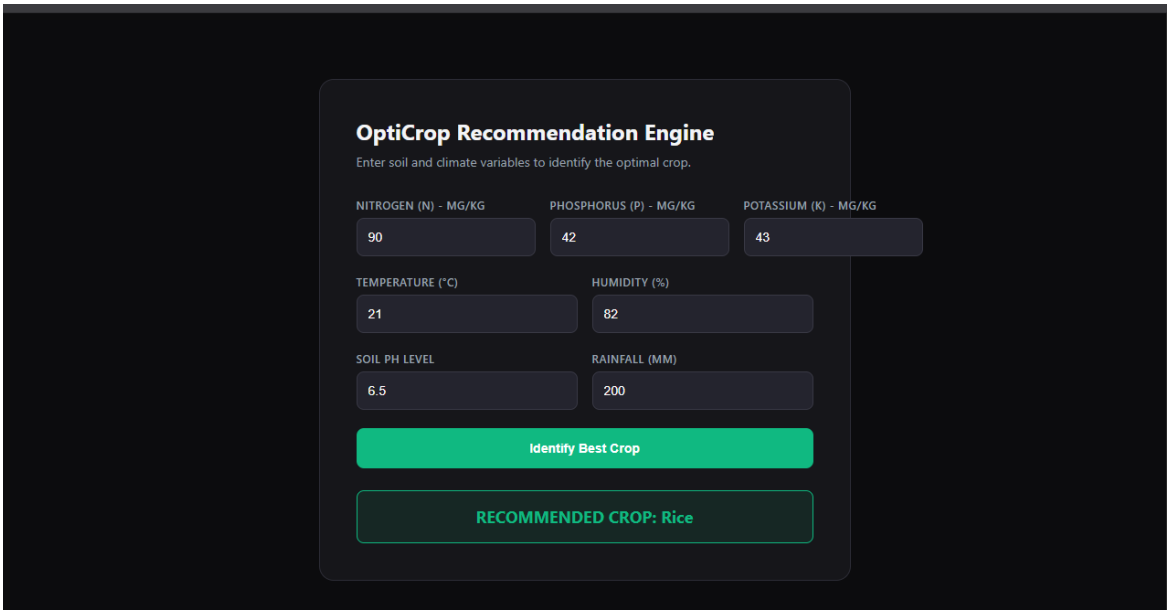
The home page is displayed upon launching the application, introducing the purpose of the Smart Agricultural Production Optimization Engine with navigation to Home, About, and FindYourCrop pages.

About Page

Clicking “About” in the navigation redirects to the About page, which explains how machine learning techniques are used to improve agricultural productivity, optimize farming decisions, and support sustainable agricultural practices.

FindYourCrop Page

Clicking "Find Your Crop" redirects to the prediction form. The user enters environmental conditions and clicks "Predict" to receive a crop recommendation.



Activity 5.3: Manual Verification and Test Scenarios

Test Scenario 1: Optimal Rice Soil Profile

- Nitrogen (N): 90
- Phosphorus (P): 42
- Potassium (K): 43
- Temperature: 21
- Humidity: 82
- pH: 6.5
- Rainfall: 200

Expected Result: "RECOMMENDED CROP: Rice" displays in a green success box.

Test Scenario 2: High-Risk Out-of-Bounds Input

- Enter a pH value of 15.0.
- Click "Identify Best Crop."

Expected Result: "Prediction Error: Soil pH must be between 0.0 and 14.0" displays in a red warning box, confirming that the Flask API validation checks are actively defending the machine learning pipeline from invalid inputs.

Activity 5.4: Common Run Issues and Fixes

Error: Address already in use (Port Conflict)

If the terminal shows OSError: [Errno 98] Address already in use, another service (such as macOS AirPlay Receiver or an orphaned Python process) is already listening on port 5000.

The Fix: Open app.py, scroll to the bottom, and change the port configuration:

```
app.run(port=5001, debug=True)
```

Then rerun python app.py and navigate to http://localhost:5001.

Error: TemplateNotFound

If the page shows jinja2.exceptions.TemplateNotFound: index.html, the HTML file is in the wrong directory.

The Fix: Ensure index.html resides inside a folder named templates. If it is in the project root or inside static, Flask's template engine will not find it.

Milestone 6: Conclusion

OptiCrop is a data-driven Smart Agricultural Production Optimization Engine built with Python, Scikit-learn, and Flask. By assessing key soil nutrients (Nitrogen, Phosphorous, Potassium) and local weather metrics (temperature, humidity, pH, and rainfall), the system predicts the highest-yielding crop for a given plot, replacing historical guesswork with data-driven recommendations.

Throughout the project, the team built an ingestion pipeline that loads and cleans soil features; a Jupyter EDA workflow validating Nitrogen, Phosphorus, and Potassium metrics, identifying column correlations, and scaling numeric ranges; a machine learning pipeline comparing Logistic Regression, K-Nearest Neighbors, Decision Tree, and Random Forest models alongside unsupervised K-Means clustering; a Flask API backend bridging raw JSON requests with high-performance model predictions and boundary-parameter validation; and an HTML/CSS/JS frontend interface displaying dynamic crop suggestions in a styled dashboard layout. The Random Forest Classifier was selected as the best-performing model with 99.32% test accuracy.

The project highlights how targeted machine learning and a structured development framework can boost productivity and decision quality for farmers, researchers, and policymakers, with strong potential for future scalability.

Future Extensions

- Feature Importance Plotting: analyze model feature weights to see which variables (such as rainfall vs. potassium levels) affect crop recommendation classifications the most.
- Weather API Integration: connect the backend to a live weather forecasting API (such as OpenWeatherMap) to fetch local temperature, humidity, and rainfall forecasts automatically.
- Cloud Deployment: package the application inside a Docker container and deploy it to a cloud provider (such as Render or AWS) to make the crop recommender accessible online to agricultural researchers.

- Multi-language Support and User Authentication: extend the platform to support multiple languages and save each farmer's generation history.